

Cognition Plane Supplement

Complement to *AI Agent Orchestration — Concept Design (Secure Control Layer)*

Purpose: Bridge production agent orchestration (hybrid routing, multi-agent state, RAG, recovery, eval) with a security-first enterprise control plane—without competing on positioning.
Audience: Technical diligence, Agentic AI Lead / Engineer interviews
Status: Draft for discussion | Confidential

1. Executive alignment (30 seconds)

Your concept design wins on **operational trust**: identity before execution, tenant isolation, policy gates, audit evidence, and workflow embedding (Jira, templates, metering).

What I have shipped in production (**QUEST** and adjacent systems) wins on **execution that survives beyond the demo**: deterministic + LLM hybrid orchestration, stateful multi-agent workflows, guardrails, RAG with measured retrieval gains, failure recovery, and observability/eval hooks.

Together: You sell *governed execution*; I bring *production-grade agent runtime* that runs **inside** your trust boundaries and feeds your telemetry, chargeback, and compliance story.

One line for the room: “You built the airport security and air traffic control; I built the aircraft and flight procedures—and I design them so every maneuver is logged, scoped, and recoverable.”

2. The gap (honest) and how to close it in conversation

Their main focus (Concept Design)	My strongest proof points	Perceived gap	How I close it
AWS micro-account + EKS tenant isolation	GCP/FastAPI production systems	Cloud substrate	Same patterns: bounded runtime, per-tenant config/secrets, network segmentation — implementation

Their main focus (Concept Design)	My strongest proof points	Perceived gap	How I close it
			detail, not architectural disagreement
Pre-execution identity & policy gates	Guardrails, injection defense, audit trails on runs	Identity product depth	I enforce at runtime what you gate at admission ; eager to implement against your identity API, not reinvent IAM
Central control plane + scheduling/ SLA	QUEST scheduling, routing, idle vs spawn semantics	Multi-tenant SaaS control plane at Fortune 500 scale	I have operated orchestration intelligence in prod; I want to embed it in your plane, not fork a competing product
Security-led GTM, compliance evidence	Eval pipelines, structured logs, reproducible runs	Buyer-facing compliance matrix	I speak engineering reliability; I want to learn your questionnaire → control mapping and package eval output as audit artifacts
Jira/web template intake	Tool-using agents across APIs and vector DBs	First-class Jira wedge	Templates map directly to routing tables + tool allowlists + output schemas in my orchestrator

Do not argue that your secure layer is optional. **Do argue** that enterprises block pilots when the secure layer exists but the **runtime** is still fragile—my work de-risks the second objection.

3. Two-plane architecture (integration, not overlap)

```
flowchart TB
    subgraph trust["Trust plane (your platform)"]
        intake["Task intake: Jira / Web"]
        tpl["Execution templates"]
        id["Identity validation"]
        pol["Policy gates"]
        sched["Scheduler: priority, SLA, metering"]
        obs["Central telemetry & audit store"]
    end

    subgraph cog["Cognition plane (my contribution)"]
        route["Hybrid router: rules + LLM"]
        graph["Stateful orchestration: graphs / FSM"]
        tools["Tool layer: APIs, DB, RAG, structured outputs"]
        guard["Runtime guardrails"]
        ckpt["Checkpoints & failure recovery"]
        eval["Eval & regression hooks"]
    end

    intake --> tpl --> id --> pol --> sched
    sched -->|"Approved run spec + scoped creds"| route
    route --> graph --> tools
    graph --> guard
    graph --> ckpt
    eval --> obs
    guard --> obs
    graph --> obs
    pol -. ->|"Tool allowlist, data scope"| tools
```

Contract between planes (what I would implement against)

Control plane emits	Cognition plane returns
<code>tenant_id</code> , <code>run_id</code> , <code>template_id</code>	Same IDs on every event
Scoped credentials + tool allowlist	Tool calls only within allowlist; fail closed
Output schema / constraints	Validated structured result or explicit failure code
SLA tier, priority	Routing choice (fast path vs deep reasoning)
Policy version	<code>policy_version</code> echoed in logs for audits

Every agent step becomes an **evidence event**: who (identity), what (tool/model), when, input hash, outcome, token/cost, eval score.

4. What I have already built (mapped to your capabilities)

4.1 QUEST — multi-agent orchestration (production)

- **Stateful orchestration layer** with checkpoints and resumable runs
- **Hybrid routing**: deterministic paths for known workflows; LLM planner only when templates permit ambiguity
- **Failure recovery**: retries, alternate paths, escalation—not silent partial success
- **Multi-agent memory** scoped per run (not a shared black box)
- **FastAPI services on GCP** with operational ownership (latency, token usage, stuck-run debugging)

Maps to your PDF: Step 3–5 (validate → route → execute → return with logs). My layer is the **orchestrator + agent runtime** behind your validation gate.

4.2 RAG and retrieval (measured, not demo)

- Pipelines with **embedding fine-tuning** (triplet loss) reporting **~25% retrieval accuracy improvement** over baseline
- Vector search integration (e.g. Azure AI Search—class patterns); same design ports to tenant-scoped indexes inside isolated runtimes

Maps to your PDF: Wedge workflows (incident enrichment, ticket triage) need **grounded** answers; retrieval quality is a pilot KPI, not a slide footnote.

4.3 Safety, reliability, evaluation

- **Guardrails** and prompt-injection awareness at runtime
- **Auditability**: structured run logs suitable for governance review
- **Eval orientation**: regression-style checks on outputs (hallucination/risk scoring as hooks into central telemetry)

Maps to your PDF: “Evidence-ready logging” and 60–90 day pilot metrics—I supply **scorable execution**, you supply **enterprise retention of evidence**.

5. Reference workflow: incident enrichment (wedge + my stack)

Aligns with your stated wedge (incident enrichment, ticket triage) and shows end-to-end fit.

Step	Trust plane	Cognition plane (my patterns)
1	Jira issue opened; template defines sources, PII rules, output format	—

Step	Trust plane	Cognition plane (my patterns)
2	Identity + policy: which repos, tickets, search indexes allowed	—
3	Scheduler assigns SLA; meter starts	Router: if template = “enrichment v2”, deterministic subgraph; else bounded planner
4	Short-lived creds issued	Agent A: retrieve logs/docs (RAG + API tools)
5	—	Agent B: synthesize structured Jira comment + severity suggestion
6	—	Guardrail pass on outbound text; eval hook scores grounding
7	Result + execution logs to customer audit store	Full trace keyed by <code>run_id</code>

Failure modes I handle: tool timeout, model refusal, partial retrieval—checkpoint and retry or escalate with explicit status (never unlogged “best effort”).

6. Roadmap handshake (their phases × my delivery)

Their roadmap phase	What I accelerate
0–6 mo — Pilot & hardening	Embed cognition plane behind one template (e.g. Jira enrichment); hybrid router + guardrails + basic eval in CI
6–12 mo — Integration expansion	MCP/tool adapters for enterprise toolchains; multi-agent pattern library; regression eval tied to governance dashboards
12–18 mo — Scale	Long-running workflows with checkpoint storage; model health scoring; optional retrieval fine-tuning per tenant policy

Fine-tuning (SFT/LoRA/RLHF/GRPO): **adjacent strength**—I have delivered embedding-level gains; full model fine-tuning is a **module** once tenant isolation and data governance are fixed (your plane first).

7. Role fit: complete package narrative

Agentic AI Lead (how I present)

- Think in **platform contracts**: trust plane vs cognition plane, long-term integrity, no bypass of pre-execution identity
- Architectural tradeoffs: LangGraph/custom FSM **in-tenant** vs shared agent pool; when deterministic routing must dominate for compliance
- Explain **reliability and safety** to executives: fail-closed, evidence per step, pilot KPIs tied to eval—not model vanity metrics

Agentic AI Engineer (how I present)

- Strong **Python**, production **ownership**, war stories on token blowups, orchestration deadlocks, and partial tool failures
- Systems that **survive beyond the demo**: QUEST in prod, not notebook agents

Top needs — direct answers to prepare

Need	My answer
Claude production	<i>[Prepare: versions, structured outputs, cost caps, fallbacks you have used]</i>
LangGraph / MCP / state machines	QUEST = stateful orchestration; MCP = integration surface I would standardize on for your roadmap
Tool-using agents + structured outputs	Core of QUEST; templates → schemas
Deterministic + LLM hybrid	Primary design pattern I ship
Eval frameworks	Hooks + regression mindset; eager to harden into your telemetry
Long-running / large context	Checkpoints + summarization + external memory; not unbounded prompt stuffing
Enterprise / distributed	Ready to operate inside your isolation model; my scale story is orchestration reliability
Healthcare / life sciences	<i>[If applicable: PHI handling, validation rigor—otherwise: regulated-enterprise parallel: audit + fail-closed]</i>

8. Interview talking points (use verbatim if helpful)

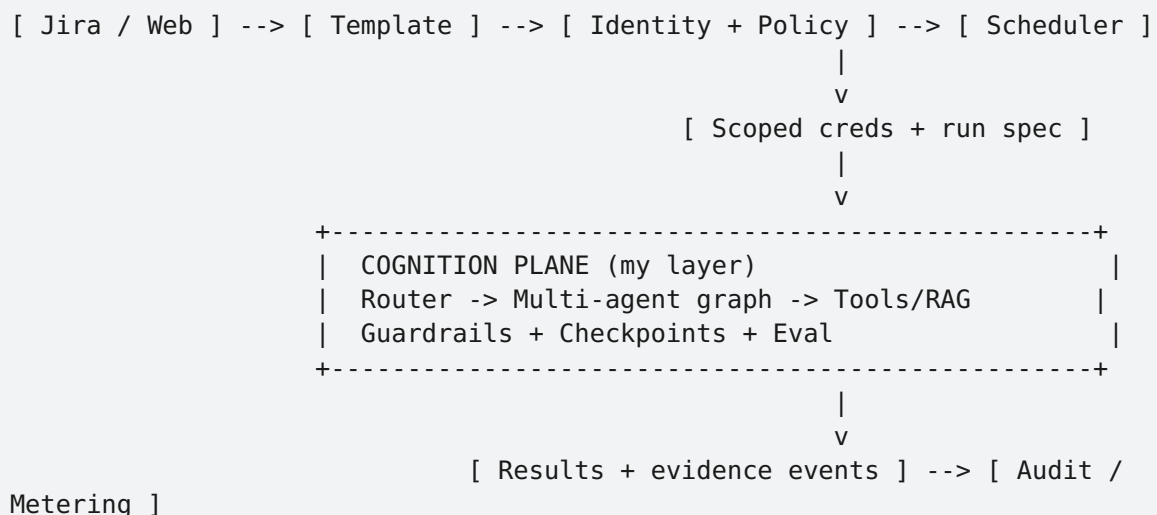
1. **“I’m not here to rebuild your moat.”** Your moat is trust architecture and workflow embedding. I’m here to make the agent runtime worthy of that trust.

2. **“Policy twice.”** You enforce policy before run; I enforce again at every tool and model boundary so audit and security teams see defense in depth.
3. **“Templates are the API.”** Your mandatory templates become my routing spec, tool allowlist, and output schema—one product surface for ops and security.
4. **“Pilots fail on runtime, not slides.”** Design partners need enrichment/triage that **completes** with logs and eval scores; that’s what I’ve shipped.
5. **“Ask me to whiteboard the seam.”** I want to draw the event contract between control plane and orchestrator—that’s the Lead job on day one.

Questions to ask them (shows platform thinking)

- What is the **minimum event schema** for audit today, and what’s missing for agentic steps (tool calls, model IDs, eval scores)?
- Where do **templates** live—product UI, Jira plugin, or API—and who owns versioning with policy?
- Pilot success metrics at 60–90 days: productivity only, or **risk reduction** (blocked actions, injection attempts, policy violations)?
- Is agent code **customer-supplied**, **platform-owned**, or marketplace—and how does that interact with EKS isolation?

9. Optional one-pager diagram (ASCII for slides or notes)



10. Closing position

They are building the category infrastructure for **trusted** autonomous execution in the enterprise.

I am building the **production cognition layer** that makes trusted execution deliver real operational outcomes.

The gap between us is **go-to-market emphasis**, not **technical incompatibility**. Packaged together: security teams get fail-closed governance; operations teams get enrichment and triage that finish with auditable artifacts; finance gets metering on real work units.

I am interviewing to **integrate** what I have built into this architecture—not to reposition the company as another agent framework vendor.

Personalize before interview: fill bracketed Claude/healthcare rows; add one QUEST metric (p95 latency, recovery rate, eval pass rate) if you can share it.